Inteligencia Artificial

Mapa del teórico — IUA 2026. Cómo se conectan los diez encuentros.

Este documento no reemplaza las diapositivas: las ordena. La materia parece saltar de estadística a algoritmos de búsqueda y después vuelve a los modelos, y en el medio es fácil perder el hilo. La idea acá es mostrar cuál es ese hilo, qué pregunta responde cada deck y en qué punto un tema vuelve a aparecer con otro nombre.

> **La pregunta que atraviesa toda la materia**
>
> Cómo construir un sistema que tome buenas decisiones. Hay dos caminos para lograrlo y la materia recorre los dos: **aprender de datos**, que es el bloque de EDA y preprocesamiento, y **razonar sobre un espacio de posibilidades**, que es el bloque de agentes, búsqueda y optimización. Los dos caminos se juntan en el último encuentro.

1. El mapa en una página

Deck	La pregunta que responde	Bloque
00. Presentación	Cómo se cursa y cómo se aprueba.	—
01. Fundamentos IA / ML	Qué es la IA, qué es el aprendizaje automático y con qué herramientas se trabaja.	Base
02. Introducción al análisis	Cuál es el flujo de trabajo con datos y qué tipos de variable existen.	A
03. EDA	Cómo describir un dataset y medir relaciones entre variables.	A
04. Faltantes y outliers	Cómo decidir si una relación es real y por qué faltan los datos que faltan.	A
05. Limpieza	Cómo dejar los datos listos: faltantes, outliers, codificación y el split.	A
06. Nuevas features	Cómo crear y transformar variables sin contaminar la evaluación.	A
07. Agentes	Cómo un agente decide, y cómo buscar una solución en un espacio de estados.	B
08. Optimización	Qué hacer cuando el camino no importa y solo importa el estado final.	B
09. Aprendizaje automático	Por qué un modelo falla al generalizar y cómo elegir el modelo correcto.	C

Los tres bloques en una frase cada uno

- **Bloque A, decks 02 a 06.** Los datos nunca vienen listos. Antes de que un modelo aprenda algo hay que entenderlos, limpiarlos y transformarlos, y hacerlo sin hacer trampa.
- **Bloque B, decks 07 y 08.** No todo problema se resuelve con datos. Si las reglas son conocidas, conviene buscar la solución en el espacio de posibilidades en vez de aprenderla.
- **Bloque C, deck 09.** Vuelve al aprendizaje, pero ahora con la pregunta difícil: no cómo ajustar bien los datos que tengo, sino cómo acertar con los que todavía no vi.

2. Bloque A — De los datos al modelo

Deck 01. Fundamentos de IA y Machine Learning

Es el encuadre. Define la IA, separa lo que es aprendizaje automático de lo que son reglas programadas a mano, y presenta las tres formas de aprender que vuelven en el deck 09.

- **Supervisado:** hay pares de entrada y salida, y el modelo aprende la relación. Se divide en clasificación, si la salida es categórica, y regresión, si es numérica.
- **No supervisado:** no hay salida conocida. El modelo descubre estructura: clustering, reglas de asociación, reducción de dimensionalidad.
- **Por refuerzo:** el agente aprende por ensayo y error a partir de recompensas y castigos.

La segunda mitad del deck es herramienta pura: Python, Jupyter, NumPy, Pandas, Matplotlib, Seaborn, Scikit-Learn, y programación orientada a objetos.

> **Dónde vuelve**
>
> El aprendizaje por refuerzo es el puente al deck 07: un agente que mejora con la experiencia. Y la división entre clasificación y regresión se retoma formalmente en el deck 09.

Deck 02. Introducción al análisis de datos

Introduce el flujo de trabajo que después se repite en casi todos los decks: identificar el problema, recopilar los datos, hacer EDA y limpieza básica, preparar y analizar, entrenar modelos. Es un proceso iterativo, no una línea recta.

Define también los tipos de variable, que condicionan todo lo que viene: qué estadístico se puede calcular, qué gráfico corresponde y qué codificación aplica.

Deck 03. Análisis exploratorio de datos

Es el deck más denso en estadística. Se organiza en dos mitades.

Describir una variable sola

- **Numéricas:** tendencia central con media, mediana y moda; dispersión con varianza y desviación estándar; forma con asimetría y curtosis.
- **Estadística robusta:** cuantiles, cuartiles y rango intercuartílico. Sirve cuando hay valores extremos que arruinan la media.
- **Categóricas:** frecuencias, proporción de la moda y entropía de Shannon.

Medir la relación entre dos variables

- **Numérica con numérica:** covarianza y correlación. Pearson mide relación lineal, Spearman y Kendall trabajan sobre rangos y detectan relaciones monótonas no lineales.
- **Con categóricas:** tablas de contingencia y gráficos comparativos.

> **Dónde vuelve**
>
> El rango intercuartílico es la regla con la que se detectan outliers en el deck 04. La entropía de Shannon reaparece en el deck 05 para medir desbalance de clases. Y la asimetría decide, en el deck 06, si conviene discretizar por intervalos iguales o por cuantiles.

Deck 04. Causas e identificación de faltantes

Responde una pregunta que el deck 03 deja abierta: una correlación alta, ¿es real o es casualidad de la muestra? Para eso introduce los tests de hipótesis, con chi-cuadrado para variables categóricas y ANOVA para comparar grupos, y las medidas de asociación: coeficiente Phi, correlación punto-biserial, V de Cramér, Tau-b de Kendall y eta cuadrado.

La segunda mitad es la clasificación de faltantes, que es lo que más se evalúa:

Tipo	Qué significa	Ejemplo del teórico
MCAR	Falta al azar, sin relación con nada. No introduce sesgo.	El formulario falla al azar.
MAR	La falta depende de **otra** columna. Hay un patrón.	Los mayores de 40 no declaran el sueldo.
MNAR	La falta depende del **valor que falta**. Difícil de detectar.	Los que ganan más no contestan.
Estructural	El dato no existe. Tiene sentido que no esté.	Año de graduación de quien no se graduó.

> **Dónde vuelve**
> El tipo de faltante decide la estrategia de imputación del deck 05. No es una clasificación teórica: cambia qué se hace con la columna.

Deck 05. Procesamiento y limpieza

Acá aparecen tres ideas que se evalúan mucho y conviene no mezclar.

El split y el desbalance

- Se parte el dataset en entrenamiento, usualmente 70 u 80 por ciento, y prueba con el resto.
- **Stratified split:** mantiene en train y test la misma proporción de clases que en el original. Es esencial con datasets desbalanceados. Importante: **no corrige** el desbalance, lo conserva.
- En series temporales se separa por fecha, para no mezclar pasado y futuro.

Imputación según el tipo de faltante

Mecanismo	Muy baja, menos de 10%	Baja, 10 a 30%	Media, 40 a 60%	Alta, 70% o más
MCAR	Eliminar filas	Imputación simple	Técnicas avanzadas	Eliminar columna
MAR	Imputación simple **considerando el patrón**	Imputación simple **considerando el patrón**	Técnicas avanzadas	Eliminar columna
MNAR	Modelado especial	Modelado especial	Modelado especial	Modelado especial
Estructural	Imputar con 0 o nueva categoría	Imputar con 0 o nueva categoría	Redefinir la variable	Redefinir la variable

Codificación según la cardinalidad

Técnica	Ideal para	Problema
One-hot	Baja cardinalidad	Escala mal con alta cardinalidad
Ordinal	Variables con orden lógico	Crea relaciones numéricas artificiales
Hashing	Cardinalidad muy alta	Colisiones, no es reversible
Frequency	Muchas categorías frecuentes	Puede perder información
Target	Alta cardinalidad, problema supervisado	Riesgo de sobreajuste
Cyclic	Variables con patrón cíclico, como meses	No sirve sin periodicidad clara
Label	Solo para la variable target	Implica orden

Deck 06. Nuevas features y el flujo correcto

Cierra el bloque A con la pieza que más se evalúa: el orden de las operaciones.

- **Feature engineering:** crear variables nuevas a partir de las que hay, por ejemplo una fecha de nacimiento que se convierte en edad, o peso y altura que se combinan en el índice de masa corporal.
- **Discretizar:** pasar de continuo a categórico. Por intervalos iguales, de igual amplitud, o por puntos de corte, donde cada intervalo tiene la misma cantidad de observaciones.
- **Escalar:** normalizar lleva al rango 0 a 1 y sirve para algoritmos de distancia como KNN; estandarizar lleva a media 0 y desvío 1 y sirve para modelos lineales, SVM y PCA.
- **Balancear:** oversampling, por ejemplo SMOTE, o undersampling.

> **La regla de oro del deck 06**
>
> Todas las transformaciones se **calculan** solo con train y después se **aplican** también a test. La única excepción es el **balanceo**, que se aplica únicamente a train y nunca al test, porque el test tiene que conservar la proporción real de clases para que la evaluación sea honesta.

El motivo de fondo es la fuga de información: si una transformación se calcula sobre el dataset completo, el entrenamiento recibe pistas del test y las métricas salen demasiado optimistas. Ese argumento es exactamente el mismo que justifica el set de validación en el deck 09.

El deck cierra con métodos avanzados de imputación: KNN, que busca los vecinos más parecidos y por eso exige normalizar antes, y MICE, que es imputación múltiple iterativa.

3. Bloque B — Agentes que deciden

Acá la materia cambia de eje y conviene notarlo. En el bloque A el conocimiento está en los datos y hay que extraerlo. En el bloque B las reglas del problema son conocidas y lo que falta es encontrar la solución dentro de un espacio de posibilidades.

Deck 07. Agentes racionales y búsqueda

Qué es un agente

Un agente percibe el entorno con sensores y actúa con actuadores. Un **agente racional** elige, para cada secuencia de percepciones, la acción que maximiza su medida de rendimiento. Para definirlo se usa **PEAS**: Performance, Environment, Actuators, Sensors. Pensar un sistema en términos de PEAS obliga a definir qué es el éxito antes de programar.

Los cinco tipos de programa de agente van de menos a más capaces:

- **Reactivo simple:** reglas si-entonces sobre la percepción actual. Solo sirve en entornos completamente observables.
- **Basado en modelos:** mantiene estado interno. Funciona en entornos parcialmente observables.
- **Basado en objetivos:** evalúa consecuencias, se pregunta qué pasa si hace tal acción.
- **Basado en utilidad:** maximiza una función de preferencia, no solo alcanza una meta.
- **Que aprende:** mejora con la experiencia. Es el puente directo al deck 09.

Resolver problemas con búsqueda

Un problema de búsqueda se formula con seis componentes: estado inicial, estados objetivo, acciones disponibles, modelo de transición, costo de acción y solución óptima. El ejemplo de la materia es la Torre de Hanoi con 5 discos, que tiene 3^5 igual a 243 estados posibles.

Algoritmo	Cómo elige el próximo nodo	Óptimo
Anchura (BFS)	Nivel por nivel, cola FIFO	Sí, si todos los costos son iguales
Costo uniforme (Dijkstra)	Menor costo acumulado g(n)	Sí, siempre
Profundidad (DFS)	La rama más profunda, pila LIFO	No
Profundidad limitada (DLS)	DFS con un límite fijo	No
Profundidad iterativa (IDDFS)	DLS subiendo el límite	Sí, con poca memoria
Voraz (Greedy)	f(n) = h(n), el que parece más cerca	No
A*	**f(n) = g(n) + h(n)**	**Sí, con heurística admisible**

Las dos propiedades que hay que saber de memoria

Una heurística es **admisible** si nunca sobreestima el costo real que falta hasta el objetivo. Es **consistente** si cumple que h(n) es menor o igual al costo de ir de n a n' más h(n'). Si una heurística es consistente y vale cero en el objetivo, entonces es admisible, y A* garantiza el camino óptimo.

Deck 08. Problemas de optimización

Cambia la premisa. En el deck 07 el camino recorrido **es** la solución. Acá el camino no importa y solo importa la configuración final, como en un sudoku resuelto. Eso habilita los algoritmos de **búsqueda local**, que evalúan un solo estado actual y se mueven a los vecinos, sin guardar el árbol. Usan poquísima memoria y escalan a espacios enormes o infinitos.

- **Gradiente descendente o ascendente discreto:** se mueve siempre al vecino que mejora. Es voraz, y por eso se atasca en mínimos locales, mesetas y crestas. Variantes para escapar: estocástico, primera opción y reinicio aleatorio.
- **Simulated annealing:** a veces acepta un vecino peor, con probabilidad dada por la distribución de Boltzmann. Cuanto más alta la temperatura, más permisivo. Así escapa de mínimos locales.
- **Búsqueda local beam:** mantiene k estados a la vez e intercambia información entre ellos, abandonando rápido las búsquedas que no progresan.
- **Algoritmos genéticos:** variante del beam estocástico donde los sucesores se generan combinando dos padres por cruce, con analogía directa a la selección natural.
- **Espacios continuos:** los sucesores son infinitos. Se resuelve discretizando el espacio o usando el gradiente, con una tasa de aprendizaje alfa que si es muy chica avanza lentísimo y si es muy grande

oscila y nunca converge.

> **La conexión más importante de todo el documento**
>
> Ese gradiente con tasa de aprendizaje alfa **es el motor con el que se entrenan los modelos** del deck 09. Lo que en el deck 08 se presenta como un algoritmo de optimización, en el deck 09 es la forma en que un modelo ajusta sus parámetros. Y alfa es un hiperparámetro, de los que el deck 09 explica cómo elegir.

4. Bloque C — El aprendizaje, en serio

Deck 09. Aprendizaje automático

Formaliza lo que el deck 01 contó por arriba. Hay una función real desconocida f que relaciona entradas con salidas, y se busca un modelo h que la aproxime. Las salidas conocidas son el **ground truth**. El rendimiento sobre el conjunto de entrenamiento es el **riesgo empírico**, y lo que importa de verdad no es ese número sino la **generalización**, es decir, qué tan bien funciona con entradas que nunca vio.

Sesgo y varianza

	Modelo demasiado simple	Modelo demasiado complejo
Qué pasa	Subajuste (underfitting)	Sobreajuste (overfitting)
En entrenamiento	Error alto	Error muy bajo, memoriza el ruido
En evaluación	Error alto	Error alto
Tipo de error	Sesgo alto, varianza baja	Sesgo bajo, varianza alta
Con otra muestra	Da casi el mismo modelo	Da un modelo muy distinto

Los dos errores son complementarios: al bajar uno sube el otro. Por eso existe un punto de equilibrio, y la curva de error total contra complejidad tiene forma de U. La regla práctica del teórico es que la cantidad de parámetros entrenables debe ser mucho menor que la cantidad de observaciones de entrenamiento.

Cómo elegir bien

- **Parámetros entrenables** son los que el modelo aprende solo. **Hiperparámetros** son los que hay que fijar antes de entrenar.
- **Set de validación:** un tercer conjunto, separado del entrenamiento, para ajustar hiperparámetros sin tocar el de evaluación final. Si se usa el de evaluación para elegir, se termina optimizando para ese conjunto y se pierde generalización.
- **Validación cruzada K-fold:** se parte en K bloques; en cada vuelta uno valida y los otros K menos 1 entrenan, y al final se promedian los errores. Aprovecha todos los datos y deja intacto el conjunto de evaluación, pero cuesta K veces más cómputo.
- **Regularización:** sin regularizar hay sobreajuste; con demasiada, subajuste. En el medio está el modelo que generaliza.

5. Los hilos que atraviesan todo

Si hay algo para llevarse de este documento es esta sección. Son los temas que aparecen en un deck y vuelven en otro con otro nombre, que es justo donde uno se pierde.

Idea	Aparece primero en	Y vuelve en
No hacer trampa al evaluar. Separar lo que se usa para aprender de lo que se usa para medir.	Deck 05, con el split, y deck 06, con la regla de calcular en train y aplicar en test.	Deck 09, con el set de validación y la validación cruzada. Es el mismo principio en otra escala.
Distancia entre observaciones. Si las escalas son distintas, la variable de rango más grande domina.	Deck 06, al normalizar antes de KNN.	Deck 07, en las heurísticas que miden cuán lejos está un estado del objetivo.
Función que evalúa qué tan bueno es un estado.	Deck 07, la heurística h(n).	Deck 08, la función objetivo f(X) que se minimiza. Es el mismo rol.
Discretizar. Pasar de continuo a un conjunto finito de opciones.	Deck 06, para agrupar una variable numérica en tramos.	Deck 08, como estrategia para atacar un espacio de búsqueda continuo.
Gradiente. Moverse en la dirección que más mejora.	Deck 08, como algoritmo de búsqueda local.	Deck 09, como el mecanismo con el que el modelo ajusta sus parámetros.
Entropía de Shannon.	Deck 03, para describir una variable categórica.	Deck 05, para detectar desbalance de clases.
Rango intercuartílico.	Deck 03, como estadística robusta.	Deck 04, como la regla para detectar outliers.
Agentes que aprenden.	Deck 01, con el aprendizaje por refuerzo.	Deck 07, como el quinto tipo de agente, y deck 09 con el aprendizaje formalizado.

6. Trampas de vocabulario

Tres palabras que significan cosas distintas según el deck. Mezclarlas es el error más común.

Sesgo

En el deck 04 significa **distorsión de la muestra**: los faltantes mal tratados introducen sesgo porque el subconjunto que queda ya no representa a la población. En el deck 09 significa **error por subajuste**: el modelo es demasiado simple para captar la estructura. Son dos conceptos distintos con el mismo nombre.

Voraz (greedy)

En el deck 07 es un **algoritmo de búsqueda** que ordena la frontera solo por h(n) e ignora el costo acumulado. En el deck 08 describe al **gradiente discreto**, que avanza al vecino que mejora sin pensar en el paso siguiente. La idea de fondo es la misma, decidir mirando solo lo inmediato, pero son algoritmos distintos.

Óptimo

En el deck 07 es el **camino de menor costo** hasta el objetivo. En el deck 08 es el **estado de mejor valor** según la función objetivo, y el camino no importa. En el deck 09 es el **punto de equilibrio** entre sesgo y varianza, donde el error total es mínimo.

7. Autoevaluación

Si podés contestar estas sin mirar, el bloque está entendido.

Bloque A

- ¿Qué diferencia hay entre MAR y MNAR, y por qué cambia la estrategia de imputación?
- Con 25 por ciento de faltantes y patrón MAR, ¿qué corresponde hacer?
- ¿Por qué el stratified split no corrige el desbalance?
- ¿Qué transformación se aplica solo a train y nunca a test, y por qué?
- Una variable categórica con 98 valores distintos, ¿con qué se codifica y por qué no con one-hot?
- ¿Cuándo conviene normalizar y cuándo estandarizar?

Bloque B

- ¿Qué es PEAS y para qué obliga a pensar antes de programar?
- ¿Qué diferencia a un agente reactivo simple de uno basado en modelos?
- ¿Cuál es la diferencia entre f(n) = h(n) y f(n) = g(n) + h(n), y qué garantiza cada uno?
- ¿Qué quiere decir que una heurística sea admisible, y qué pasa si no lo es?
- ¿Cuándo conviene búsqueda local en lugar de búsqueda sistemática?
- ¿Por qué el gradiente discreto se atasca, y cómo lo evita simulated annealing?

Bloque C

- Un modelo con error bajo en train y alto en test, ¿qué le pasa y de qué tipo es su error?
- ¿Por qué no alcanza con el conjunto de evaluación para elegir hiperparámetros?
- ¿Qué resuelve la validación cruzada K-fold y cuál es su costo?
- ¿Por qué los parámetros entrenables tienen que ser muchos menos que las observaciones?

Elaborado a partir de las diapositivas de la materia Inteligencia Artificial, CRUC-IUA 2026, a cargo del Ing. González H. Alexis. Es un resumen de estudio de uso personal: ante cualquier diferencia, vale lo que dicen las diapositivas originales.